2-1 数制与编码

1.进制转换

1.1进位计数制

- d可以是0～9的任意一个，称为基数
- 10^i称为第i位上的权
- 对于二进制来说，基数为2，第i位上的权是2^i

1.2十进制与二进制的转换

- 逢十进一

2^0	1
2^1	2
2^2	4
2^3	8
2^4	16
2^5	32
2^6	64
2^7	128
2^8	256
2^9	512
2^{10}	1024
2^{11}	2048

2^{12}	4096
2^{13}	8192
2^{14}	16384
2^{15}	32768
2^{-1}	0.5
2^{-2}	0.25
2^{-3}	0.125
2^{-4}	0.0625
2^{-5}	0.03125

1.3二进制与十六进制的转换

- 逢十六进一
- 整数部分从低位向高位方向每4位用一个等值的十六进制数来替换，最后不足4位时在高位补0凑满4位
- 小数部分从高位向低位方向每4位用一个等值的十六进制数来替换，最后不足4位时在低位补0凑满4位

十进制	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
十六进制	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
二进制	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111

1.4十进制与十六进制的转换

16^0	1
16^1	16
16^2	256
16^3	4096
16^4	65536
16^5	1048576

16^{-1}	$\frac{1}{16}$
16^{-2}	$\frac{1}{256}$
16^{-3}	$\frac{1}{4096}$

2.定点与浮点表示

2.1定点表示

- 定点表示法用来对定点小数和定点整数进行表示
- 对于定点小数，其小数点总是固定在数的最左边，一般用来表示浮点数的尾数部分
- 对于定点整数，其小数点总是固定在数的最右边，用来表示整数

2.2浮点表示

- 定义与概念
  - 对于任意一个二进制数X，可以表示成如下形式：
    $X = (-1)^s \times M \times R^E$
  - 数符S：　取值为0或1，用来决定数X的符号
  - 尾数M：　一个二进制定点小数
  - 阶/指数E：一个二进制定点整数
  - 基数R：　可以取值2，4，16等
- 性质
  - 基数R一定的情况下，尾数M的位数反映数X的有效位数，它决定了数的表示精度
  - 有效位数越多，表示的精度就越高
  - 指数E的位数决定数X的表示范围，其值确定了小数点的位置
  - 任意一个浮点数都可以用一个定点整数和一个定点小数来表示

3.定点数的编码表示

概念

- 真值：一个数的真实大小
- 机器数：这个数在计算机里実际存成的二进制编码
- 无符号数：所有二进制位都用来表示数值，没有符号位 — $0 \le$ n位无符号数 $\le 2^n - 1$

3.1原码表示法

- 原码：符号位+数值位
- 当x为正数时：$x_n=0$，$x_i=x_i'$ $(0 \le i \le n-1)$
  当x为负数时：$x_n=1$，$x_i=x_i'$ $(0 \le i \le n-1)$
- 原码0有两种表示形式：
  $[+0]_{原} = 000\ldots0$
  $[-0]_{原} = 100\ldots0$

真值	-10D	-0.101B	AH
原码（二进制版）	1000 1010B	1101 0000B	0000 1010B
原码（十六进制版）	8AH	D0H	0AH

- 加减运算法则
  - 如果是两个异号数相加，则必须判定两个数的绝对值大小
  - 根据判断结果决定结果符号，并用绝对值大的数减去绝对值小的数
  - 现代计算机不用原码表示整数，只用定点原码小数来表示浮点数的尾数部分
- 表示范围 — $-2^{n-1}+1 \le$ n位原码 $\le 2^{n-1}-1$

3.2反码表示法

- 反码：反码仅作为原码和补码之间的媒介
- 对于正数：反码与原码相同
- 对于负数：原码的符号位不变，数值位取反，即可得到反码

真值	-10D	-0.101B	AH
原码	1000 1010B	1101 0000B	0000 1010B
反码	1111 0101B	1010 1111B	0000 1010B

3.3补码表示法

- 补码：补码可以实现加减运算的统一，即用加法实现减法运算
- 对于正数：补码与原码、反码相同
- 对于负数：原码的符号位不变，数值位取反，末位+1（或，反码末位+1）

真值	-10D	-0.101B	AH
原码	1000 1010B	1101 0000B	0000 1010B
补码	1111 0110B	1011 0000B	0000 1010B

- 补码真理：
  $1111\ 0110 = -2^7+2^6+2^5+2^4+2^2+2^1 = -10$
- 模运算：数到某个上限以后，不是继续往前排，而是绕回开头重新数
  - 对于某一确定的模，某数A减去小于模的另一数B，可以用A加上-B的补码来代替
  - $[x]_{补}+[y]_{补}=[x+y]_{补}$
    $[x]_{补}-[y]_{补}=[x]_{补}+[-y]_{补}$
- 补码0只有一种表示形式：
  $[0]_{补} = 000\ldots0$
- 表示范围 — $-2^{n-1} \le$ n位补码 $\le 2^{n-1}-1$

3.4移码表示法

- 移码：为了在比较两个不同符号的数时更加直观，可将每个数都加上同一个正整数转换为两个正整数再进行比较
- 移码的目的就是让[−2^(n-1), 2^(n-1)−1]这个范围内的数全部落在0的右边，变成正整数
- 移码0只有一种表示形式：
  $[0]_{移} = 100\ldots0$
- 表示范围
  - $0 \le$ 移码本身 $\le 2^n - 1$
  - $-2^{n-1} \le$ n位移码 $\le 2^{n-1}-1$

4.C语言中的类型转换

4.1C语言中的整数类型

- 短整型short，长度16位
- 整型int，长度32位
- 长整型long，长度32/64位
- 字符型char，长度8位

4.2类型转换

- 位截断 — 长类型转换为短类型时，丢弃高位，保留低位
- 位扩展
  - 短类型转换为长类型时，根据数据类型填充高位
  - 零扩展：用于无符号数，高位补0
  - 符号扩展：用于补码表示的有符号数，高位补符号位
- 相同字长转换时仅需改变解释方式

by joshuji 由 幕布 发布